

Taller: Java Backend Avanzado

July Jairetsell Leon Salcedo 20242678016

Jean Paul Toloza Cuellar 20242678009

Estrategia de Concurrencia:

Con el objetivo de cumplir con la regla de negocio principal que consiste en evitar la sobreventa de productos hemos tomado las siguientes decisiones:

- *Control de Concurrencia Optimista:* Se usa la anotación de `@Version` en las entidades de producto e inventario de forma que al ejecutar la query podamos prevenir condiciones de carrera cuando varios clientes intenten comprar un mismo artículo de forma simultánea.
- *Gestión Transaccional:* La aplicación de `@Transactional` en la capa de servicios nos permite garantizar que tanto la validación del inventario y como el descuento de las existencias a medida que se ejecutan las órdenes se realicen como una sola unidad de trabajo, otorgándole persistencia.
- *Pruebas de Carga:* Como parte de los requerimientos del taller y para comprobar el funcionamiento de la estrategia realizamos una prueba de carga con transacciones concurrentes donde simulamos múltiples solicitudes de compra, superando el inventario disponible. De esta forma verificamos que el sistema rechace transacciones donde el stock es inexistente.

Estrategia de Seguridad:

En nuestro caso partimos del modelo de Zero Trust y control de acceso por roles, por lo que se hace necesario implementar lo siguiente:

- *Autenticación:* Se realiza mediante JWT, lo que permite que la API sea stateless y pueda escalar si así se requiere.
- *Autorización:* Se definen roles específicos desde el diseño de software de forma que sea posible restringir operaciones en función de los privilegios asignados a cada rol.
- *Protección de Credenciales:* Haciendo uso de algoritmos de hashing y almacenamiento seguro de credenciales para evitar la identificación de los mismos.
- *Auditoría y No Repudio:* Realizamos un registro obligatorio no modificable de eventos de interés incluyendo intentos de inicio de sesión y accesos no autorizados para cubrir monitoreo SOC

Arquitectura de Software: Por Capas

Seleccionamos esta arquitectura para permitirnos diferenciar las responsabilidades y facilitar el mantenimiento y la escalabilidad del sistema con una base de negocio relativamente sencilla y sin la complejidad técnica que implicaría la implementación de una arquitectura hexagonal. Nuestra estructura se divide en las siguientes capas:

- **Presentación:** En esta capa gestionamos los endpoints HTTP las validaciones de entrada y los controllers.
- **Servicios:** Contiene la lógica principal, como la gestión de inventario, la administración de productos y el procesamiento de órdenes. Es aquí donde se define la frontera transaccional mediante `@Transactional` para asegurar la atomicidad de las operaciones.
- **Datos:** Implementamos los repositorios mediante Spring Data JPA y gestionamos la comunicación con la base de datos PostgreSQL.
- **Modelos:** Son las entidades transversales que representan los objetos de negocio y aquí hacemos uso de Java records para la transferencia de datos (DTOs)

De acuerdo con lo anterior, nuestra propuesta busca satisfacer los requerimientos técnicos y funcionales del taller, estableciendo una base sólida y escalable para el futuro del negocio.